UNIVERSITÀ DI BOLOGNA

School of Engineering

Master Degree in Automation Engineering

Optimal Control

Optimal Control of a Quadrotor with Suspended Load

Professor: **Giuseppe Notarstefano**

Students:GUTU ABEYA TEFERA
DEBADIPTA JHA
ZERUN HUI

Academic year 2023/2024

Abstract

KEY WORDS: OPTIMAL CONTROL, TRAJECTORY OPTIMIZATION, NEWTON'S METHOD, MPC ALGORITHM, QUADROTOR.

In this project we need to solve an optimal control problem of a quadrotor with suspended load and non-linear dynamics. The quadrotor is required to carry a load and ascend smoothly from an initial altitude to a specified second altitude and subsequently maintain a stable at the second altitude. To characterize the transition between the two altitudes, the quadrotor's states will be optimized relative to the desired values. Utilizing Newton's Method enables us to calculate the optimal trajectory from one equilibrium to another. Subsequently, a refined trajectory is established through the application of Newton's Method for a step ahead. Following this, an optimal feedback controller is formulated to follow the optimal trajectories obtained by solving a Linear Quadratic Problem (LQP). Finally, we also need to employ an MPC algorithm to track the optimal trajectories.

Contents

Introduction

Contributions

Each of us is responsible for the following:

- GUTU ABEYA TEFERA: Derivation of the quadrotor dynamic equations and gradient, code writing about trajectory generation and tracking via LQR.
- DEBADIPTA JHA: Code writting about the trajectory tracking via MPC algorithm.
- ZERUN HUI: problem setup, report writing and code debugging.

Chapter 1

Task 0 - Problem setup

1.1 System Inputs

As shown in Figure 1.1, the inputs of this simplified quadrotor model are the forces generated by the propellers.

So we have:

$$u_0 = F_s \tag{1.1}$$

$$u_1 = F_d \tag{1.2}$$

where

$$F_s = F_l + F_r \tag{1.3}$$

$$F_d = F_r - F_l \tag{1.4}$$

1.2 Discretize the Dynamics

The dynamic equations are discretized in the following way:

$$x_{0,t+1} = x_{0,t} + \delta t[V_x] \tag{1.5}$$

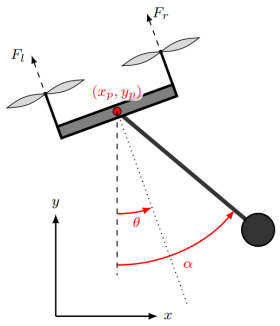

Figure 1.1: Simplified quadrotor model

$$x_{1,t+1} = x_{1,t} + \delta t[V_y] \tag{1.6}$$

$$x_{2,t+1} = x_{2,t} + \delta t[\omega_\alpha] \tag{1.7}$$

$$x_{3,t+1} = x_{3,t} + \delta t[\omega_\theta] \tag{1.8}$$

$$x_{4,t+1} = x_{4,t} + \delta t[\frac{m}{m+M} L(\omega_\alpha)^2 sin(\alpha) - \frac{F_s}{m+M}(sin(\theta) - \frac{m}{M} sin(\alpha-\theta)cos(\alpha))] \tag{1.9}$$

$$x_{5,t+1} = x_{5,t} + \delta t[-\frac{m}{m+M} L(\omega_\alpha)^2 cos(\alpha) + \frac{F_s}{m+M}(cos(\theta) + \frac{m}{M} sin(\alpha-\theta)sin(\alpha) - g)] \tag{1.10}$$

$$x_{6,t+1} = x_{6,t} + \delta t[-\frac{F_s}{ML} sin(\alpha - \theta)] \tag{1.11}$$

$$x_{7,t+1} = x_{7,t} + \delta t[\frac{lF_d}{J}] \tag{1.12}$$

where:

x_0 represents the horizontal coordinates of the quadrotor at the current time t.

x_1 represents the vertical coordinates of the quadrotor, the altitude y at the current time t.

x_2 represents the angle of the pendulum α with respect to the inertial frame at the current time t.

x_3 represents the the roll angle of the quadrotor θ at the current time t.

x_4 represents the horizontal velocity of the quadrotor V_x at the current time t.

x_5 represents the vertical velocity of the quadrotor V_y at the current time t.

x_6 represents the angular velocity of the pendulum ω_α at the current time t.

x_7 represents the angular velocity of the quadrotor ω_θ at the current time t.

δt represents the discretization time-step.

To apply Newton's Method, it is better to express the system dynamics as a vector field, as shown below. Subsequently, computing the first-order derivatives with respect to all states and inputs facilitates the derivation of the gradient for the method. And the discretized dynamic equations can be

rewritten in the following form:

$$f(x,u) = \begin{bmatrix} f_1(x,u) \\ f_2(x,u) \\ f_3(x,u) \\ f_4(x,u) \\ f_5(x,u) \\ f_6(x,u) \\ f_7(x,u) \\ f_8(x,u) \end{bmatrix} = \begin{bmatrix} x_{0,t} + \delta t[V_x] \\ x_{1,t} + \delta t[V_y] \\ x_{2,t} + \delta t[\omega_\alpha] \\ x_{3,t+1} = x_{3,t} + \delta t[\omega_\theta] \\ x_{4,t} + \delta t[\frac{m}{m+M} L(\omega_\alpha)^2 sin(\alpha) - \frac{F_s}{m+M}(sin(\theta) - \frac{m}{M} sin(\alpha-\theta)cos(\alpha))] \\ x_{5,t} + \delta t[-\frac{m}{m+M} L(\omega_\alpha)^2 cos(\alpha) + \frac{F_s}{m+M}(cos(\theta) + \frac{m}{M} sin(\alpha-\theta)sin(\alpha) - g)] \\ x_{6,t} + \delta t[-\frac{F_s}{ML} sin(\alpha-\theta)] \\ x_{7,t} + \delta t[\frac{lF_d}{J}] \end{bmatrix}$$

1.3 Get Equilibria

We explored the equilibrium points of the quadrotor system by equating the vector field describing the system dynamics to zero. Due to the non-linear nature of the problem, multiple solutions exist. The simplest solution to get is the conditions for fixed position. So we get two equilibrium points from one fixed position to another fixed position which are $[\mathbf{0\ 0\ 0\ 0\ 0\ 0\ 0\ 0}]^\top$ and $[\mathbf{5\ 5\ 0\ 0\ 0\ 0\ 0\ 0}]^\top$. In this way we defined the Step trajectory of task 1.

For task 2, to generate a desired (smooth) state-input curve, we use the bell function to create a smooth reference curve from the state $[\mathbf{0\ 0\ 0\ 0\ 0\ 0\ 0\ 0}]^\top$ to the state $[\mathbf{5\ 5\ 0\ 0\ 0\ 0\ 0\ 0}]^\top$.

Chapter 2

Task 1 - Trajectory generation (I)

2.1 Newton's Method with Affine Cost and Dynamics

In this project we use the descent algorithm usually used in optimization which is called the Newton's Method. It is more efficient and at the same time more computationally expensive than the standard Gradient Method. Considering in same cases (especially at the first iterations) the matrices Q_t^k, R_t^k and S_t^k may not be positive definite, a different version of the algorithm will be applied which is called Newton's with affine cost and dynamics. This resembles a first-order method (with only first derivatives) so it will be faster.

And for each iteration, we are required to solve the following LQR problem to compute the descent direction Δu_0, ..., Δu_{T-1}.

$$\min_{\substack{\Delta\tilde{x}_1,\ldots,\Delta\tilde{x}_T \\ \Delta u_0,\ldots,\Delta u_T-1}} \sum_{t=0}^{T-1} \frac{1}{2} \begin{bmatrix} \Delta\tilde{x}_t \\ \Delta u_t \end{bmatrix}^\top \begin{bmatrix} \tilde{Q}_t & \tilde{S}_t^\top \\ \tilde{S}_t & \tilde{R}_t \end{bmatrix} \begin{bmatrix} \Delta\tilde{x}_t \\ \Delta u_t \end{bmatrix} + \frac{1}{2}\Delta\tilde{x}_T^\top Q_T \Delta\tilde{x}_T$$

$$\text{s.t.} \Delta\tilde{x}_{t+1} = \tilde{A}_t\Delta\tilde{x}_t + \tilde{B}_t\Delta u_t$$

$$\Delta\tilde{x}_0 \textit{ given}$$

This is a LQR optimal problem by augumenting the state as $\Delta\tilde{x}_t$ and rewriting the cos and system matrices as $\tilde{Q}_t$, $\tilde{S}_t$, $\tilde{R}_t$, $\tilde{A}_t$ and $\tilde{B}_t$.

Where,

- $\Delta\tilde{x}_t := \begin{bmatrix} 1 \\ \Delta x_t \end{bmatrix}$

- $\tilde{Q}_t := \begin{bmatrix} 0 & q_t^\top \\ q_t & Q_t \end{bmatrix}$
- $\tilde{S}_t := \begin{bmatrix} r_t & S_t \end{bmatrix}$
- $\tilde{R}_t := R_t$
- $\tilde{A}_t := \begin{bmatrix} 1 & 0 \\ c_t & A_t \end{bmatrix}$
- $\tilde{B}_t := \begin{bmatrix} 0 \\ B_t \end{bmatrix}$

During each iteration, it is necessary to calculate a series of matrices and vectors for solving the LQ problem with affine terms. This process involves working backward, commencing at time t=T while taking into account $P_T = Q_T$ and $p_T = q_T$.

Where,

- $K_t^* = -(R_t + B_t^\top P_{t+1} B_t)^{-1}(S_t + B_t^\top P_{t+1} A_t)$
- $\sigma_t^* = -(R_t + B_t^\top P_{t+1} B_t)^{-1}(r_t + B_t^\top p_{t+1} + B_t^\top P_{t+1} c_t)$
- $p_t = q_t + A_t^\top p_{t+1} + A_t^\top P_{t+1} c_t + K_t^{*\top}(R_t + B_t^\top P_{t+1} B_t)\sigma_t^*$
- $P_t = Q_t + A_t^\top P_{t+1} A_t - K_t^{*\top}(R_t + B_t^\top P_{t+1} B_t) K_t^*$

And the optimal solution of the problem reads

$$\Delta u_t^* = K_t^* \Delta x_t^* + \sigma_t^*$$

$$\Delta x_{t+1}^* = A_t \Delta x_t^* + B_t \Delta u_t^*$$

At the same time we need to update the input trajectory as following:

$$u_t^{k+1} = u_t^k + \gamma^k(\sigma_t^k + K_t^k \Delta x_t^k) \approx u_t^k + \gamma^k(\sigma_t^k + K_t^k(x_t^{t+1} - x_t^k))$$

where γ^k is the step-size chosen by Armijo rule.

2.2 Step Function

In this task, we try to define a Step function between two equilibrium points for the trajectory of the quadrotor. The Step Function is generated by the file `step_reference` that simply takes constant values of the horizontal coordinate $x_p = 0$, the vertical coordinate $y_p = 0$ and the first input $F_s = 0$ for $t = \{1, 2, ..., \frac{T}{2}\}$ as well as the horizontal coordinate $x_p = 5$, the vertical coordinate $y_p = 5$ and the first input $F_s = (m + M)g$ for $t = \{\frac{T}{2} + 1, ..., T\}$. The Step Function jumps from the two levels can be defined as following:

$$x_p = \begin{cases} x_p = 0 & for \quad t = \{0, 1, 2, ..., \frac{T}{2}\} \\ x_p = 5 & for \quad t = \{\frac{T}{2} + 1, ..., T\} \end{cases}$$

$$y_p = \begin{cases} y_p = 0 & for \quad t = \{0, 1, 2, ..., \frac{T}{2}\} \\ y_p = 5 & for \quad t = \{\frac{T}{2} + 1, ..., T\} \end{cases}$$

$$F_s = \begin{cases} F_s = 0 & for \quad t = \{0, 1, 2, ..., \frac{T}{2}\} \\ F_s = (m + M)g & for \quad t = \{\frac{T}{2} + 1, ..., T\} \end{cases}$$

$$x^{des}_{0,t+1} = x_p \tag{2.1}$$

$$x^{des}_{1,t+1} = y_p \tag{2.2}$$

$$x^{des}_{2,t+1} = 0 \tag{2.3}$$

$$x^{des}_{3,t+1} = 0 \tag{2.4}$$

$$x^{des}_{4,t+1} = 0 \tag{2.5}$$

$$x^{des}_{5,t+1} = 0 \tag{2.6}$$

$$x^{des}_{6,t+1} = 0 \tag{2.7}$$

$$x^{des}_{7,t+1} = 0 \tag{2.8}$$

$$u^{des}_{0,t+1} = F_s \tag{2.9}$$

$$u^{des}_{1,t+1} = 0 \tag{2.10}$$

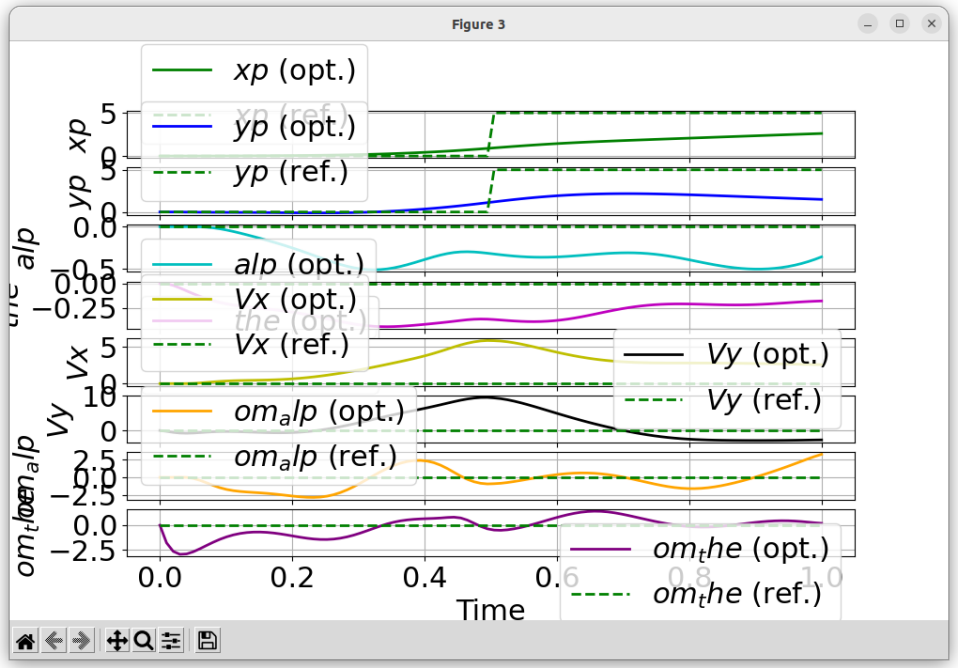

Figure 2.1: Optmal trajectory and desired curve iteration-5

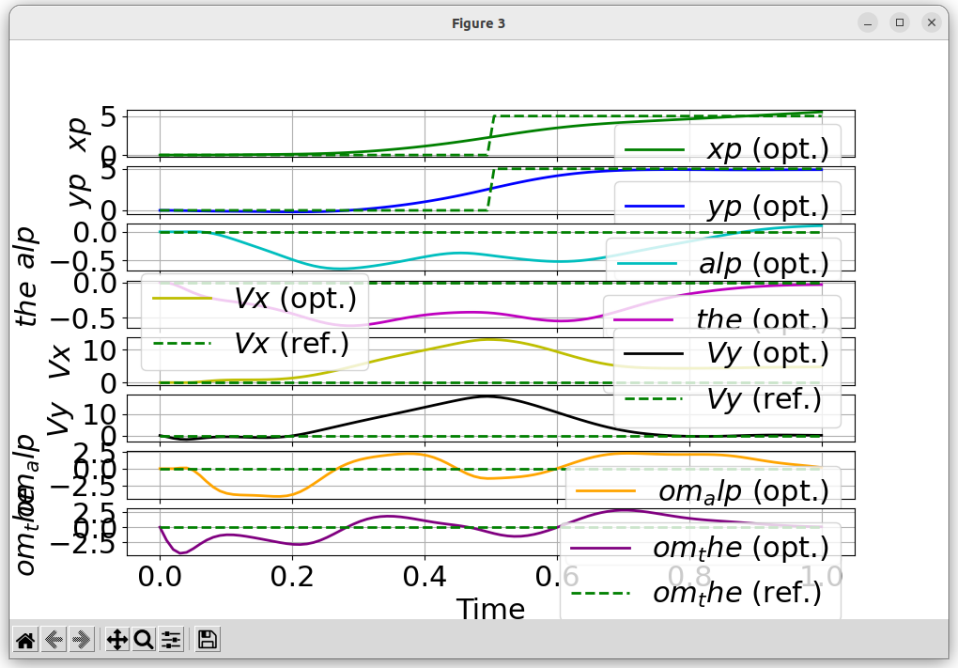

Figure 2.2: Optmal trajectory and desired curve final

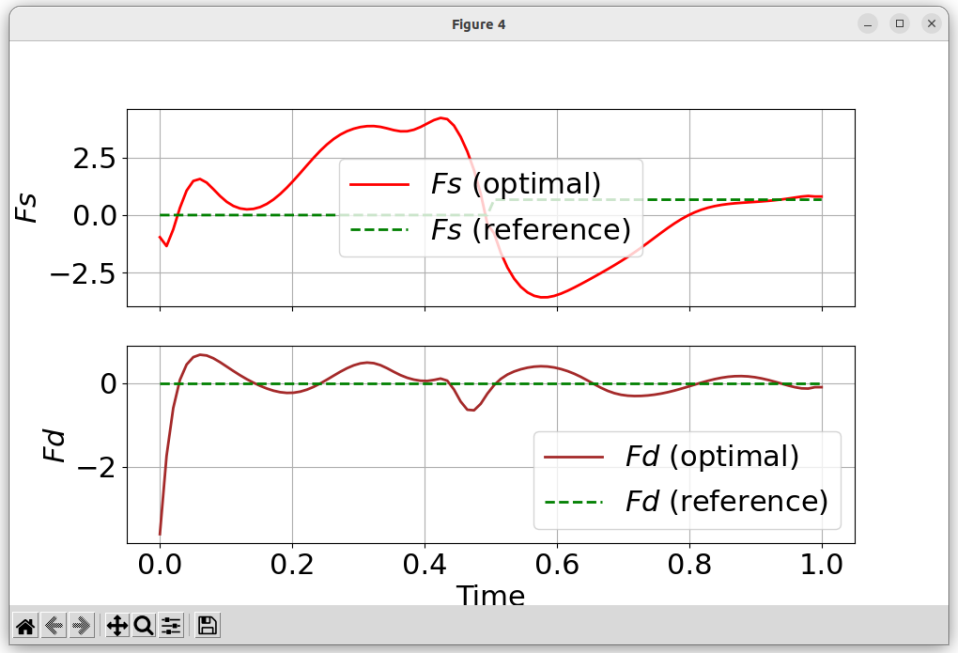

Figure 2.3: Optimal trajectory for inputs and desired input curve

As shown in Figure 2.1-2.3: the dotted lines represent the step trajectory that we defined (the reference trajectory) and the solid lines represent the optimal trajectory computed by the Newton's Method.

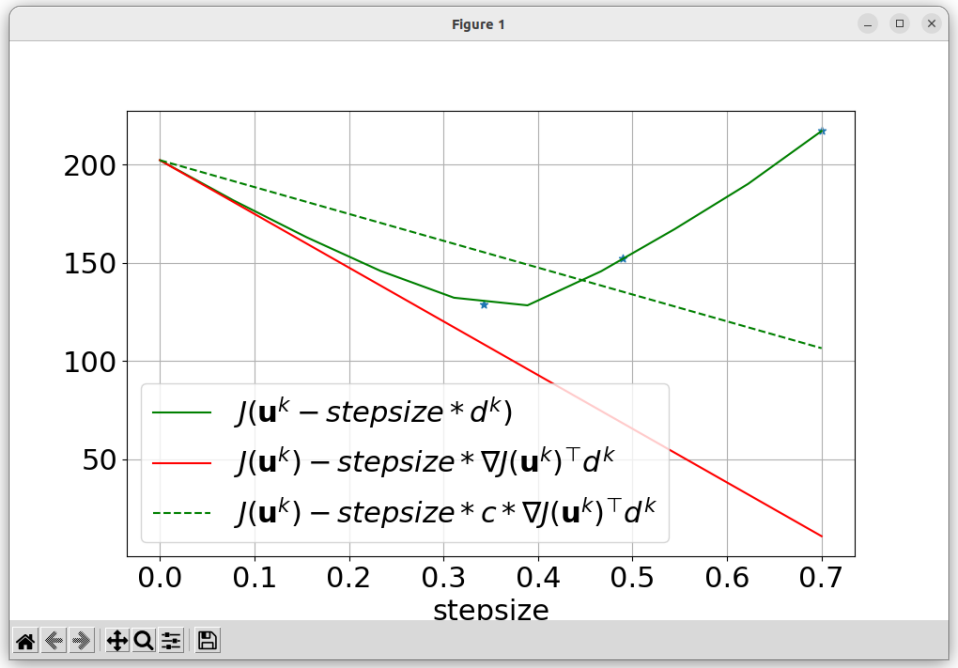

Figure 2.4: Armijo descent direction plot iteration-1

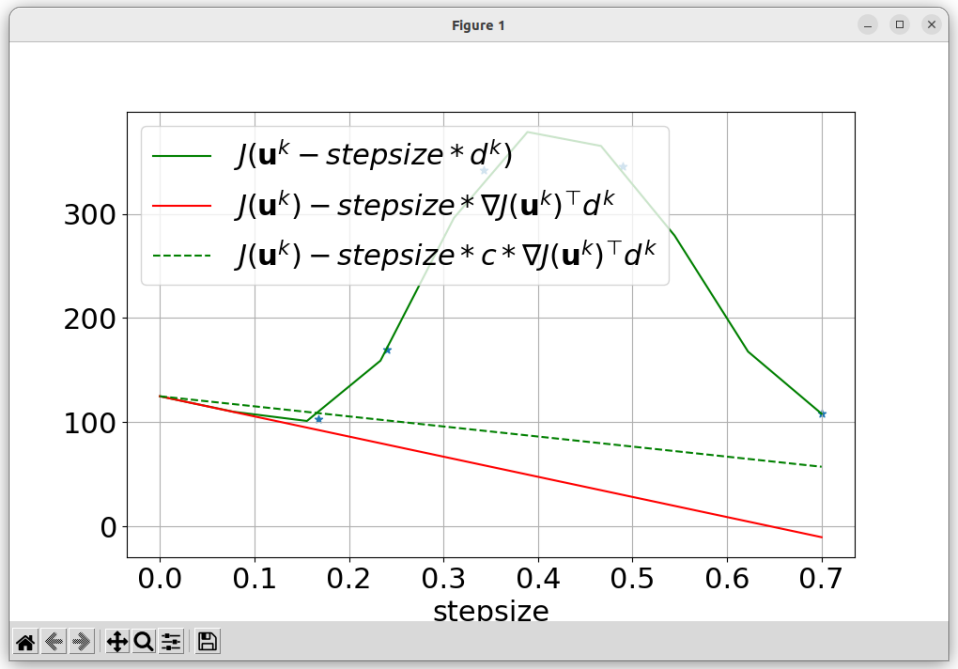

Figure 2.5: Armijo descent direction plot iteration-3

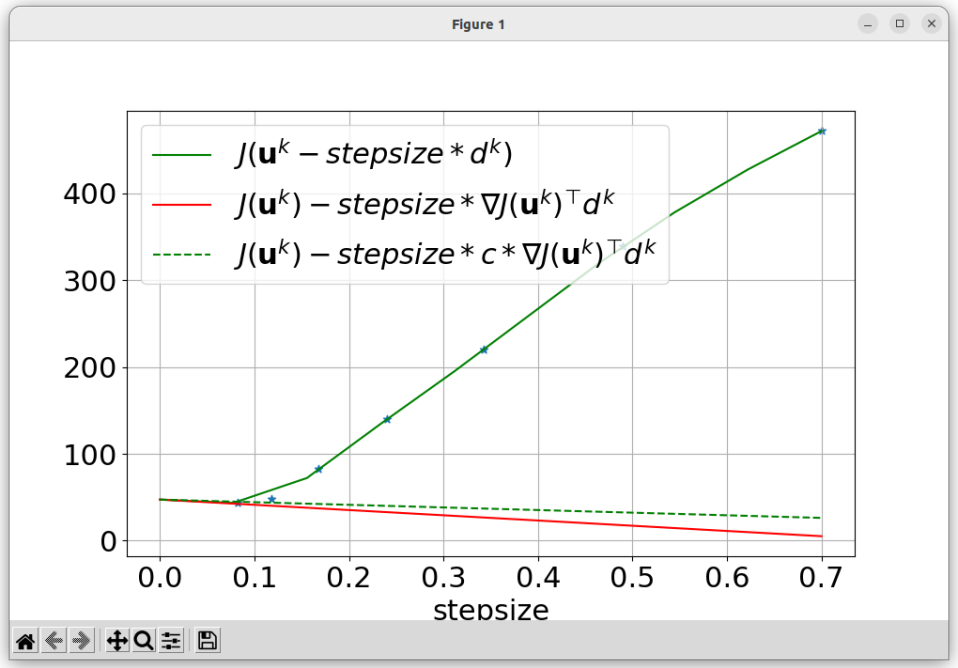

Figure 2.6: Armijo descent direction plot iteration-5

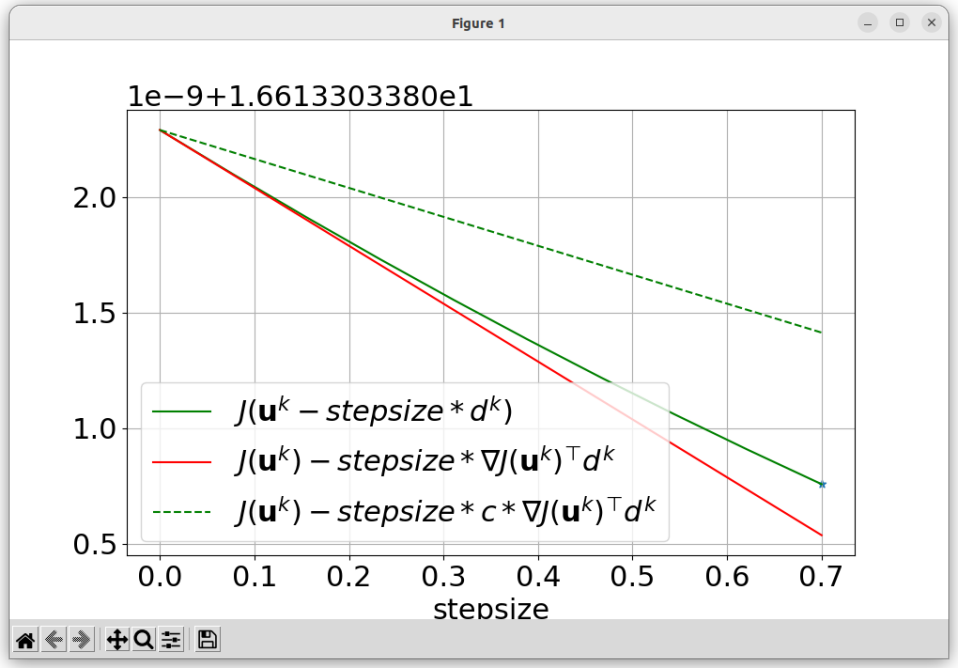

Figure 2.7: Armijo descent direction plot final

The final Armijo's selection rule and its iterations for the our Step func-

tion are shown in Figure 2.4-2.7.

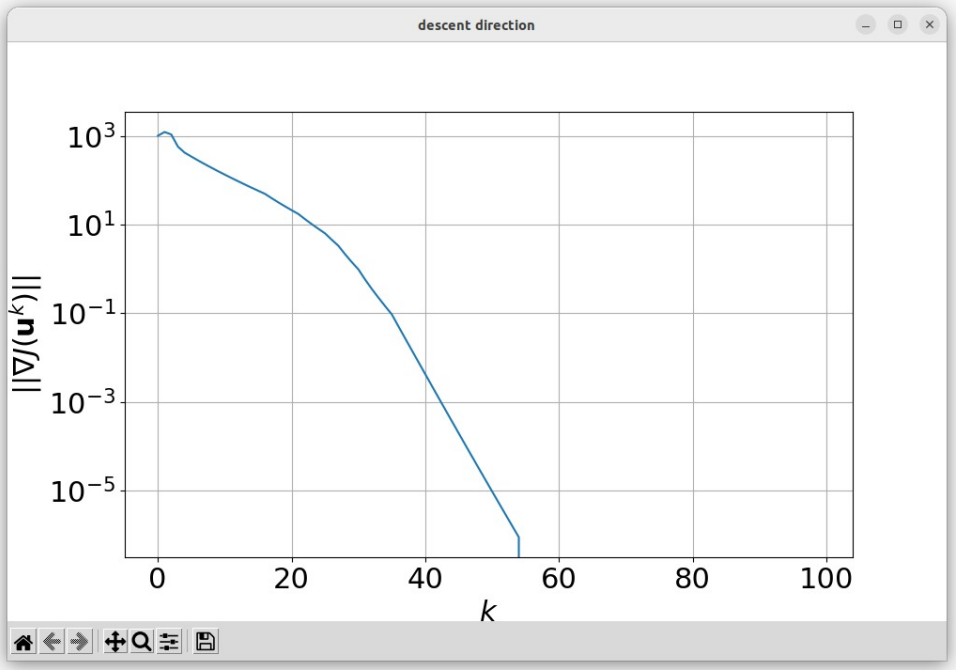

Figure 2.8: Norm of the Descent Direction

Figure 2.8 illustrates the descent direction norm for the Step trajectory. In this particular test, we established a maximum iteration limit of 100. As depicted in the plot, the descent direction norm consistently diminishes throughout the iterations. This observation indicates that our optimizer is functioning appropriately, steadily approaching the minimum.

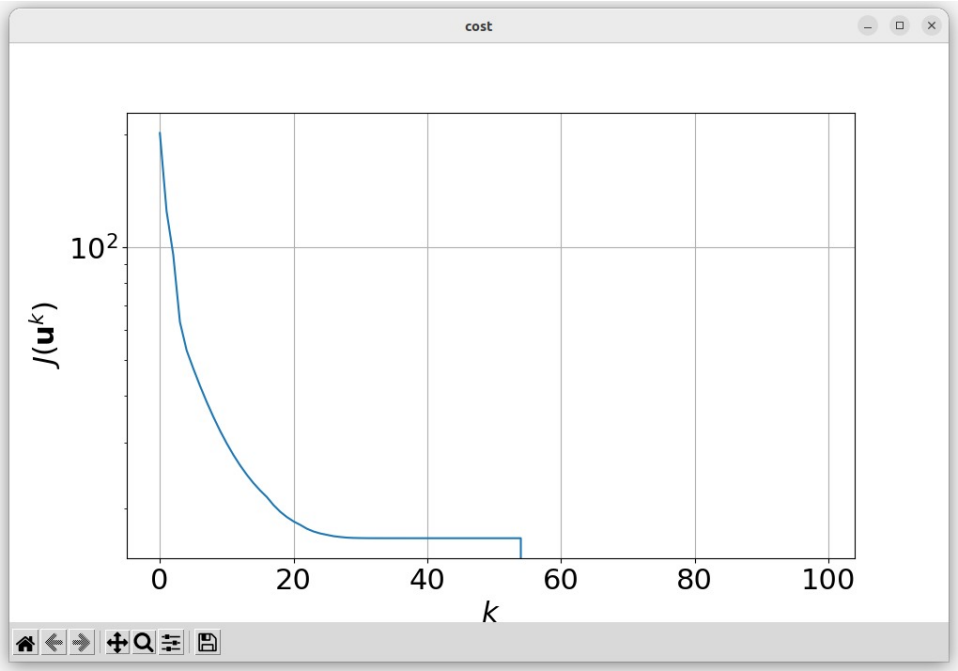

Figure 2.9: Cost of the Step Trajectory

The Cost Function continuously decreases over the course of all 54 iterations, which is a positive outcome given that our objective is to minimize it.

Chapter 3

Task 2 - Trajectory generation (II)

3.1 Bell-Shaped Reference Trajectory

In task 2, we need to generate a desired (smooth) state-input curve. As a result, we use the function `reference_curve` to define a bell-shaped trajectory. The first input F_s will be increased from 0 to (m+M)g in a smooth curve (bell-function). And meanwhile the first two states will also be increased from $[\mathbf{0}\ \mathbf{0}]^\top$ to $[\mathbf{5}\ \mathbf{5}]^\top$ in the same smooth curve (bell-function).

It should be emphasized that the trajectory with a bell-shaped curve is inherently smooth, as we utilize its mathematical definition to calculate the reference values for optimization. Here are the behaviours of the generated trajectory:

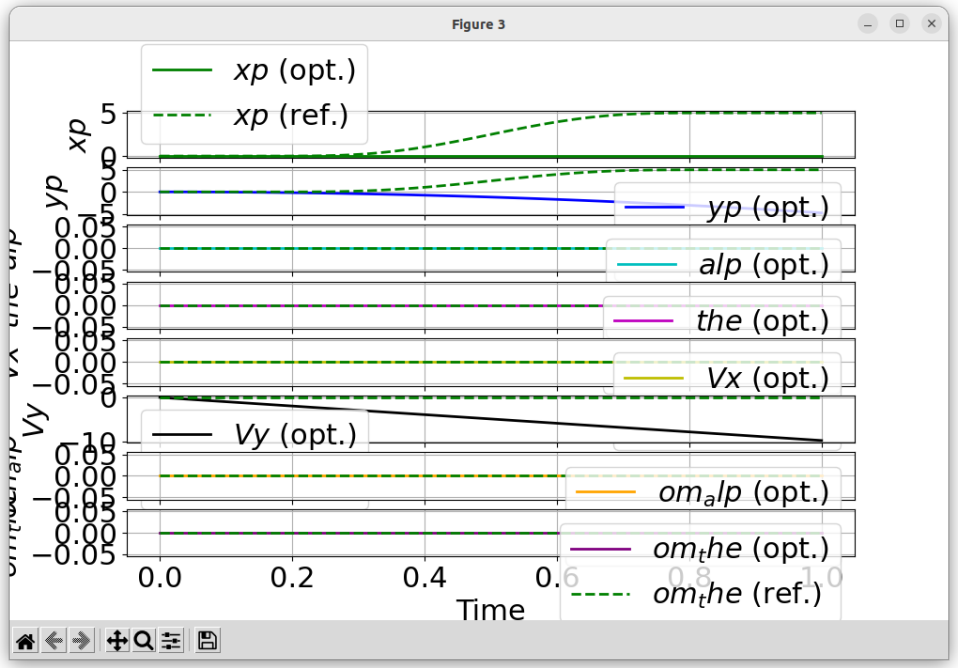

Figure 3.1: Optmal trajectory and desired curve iteration-1

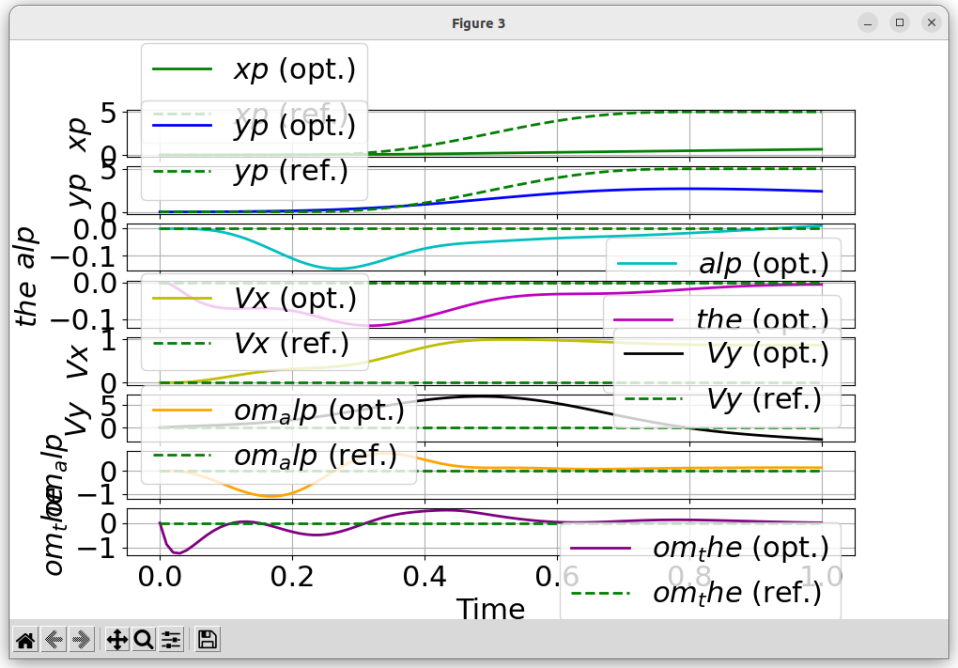

Figure 3.2: Optmal trajectory and desired curve iteration-3

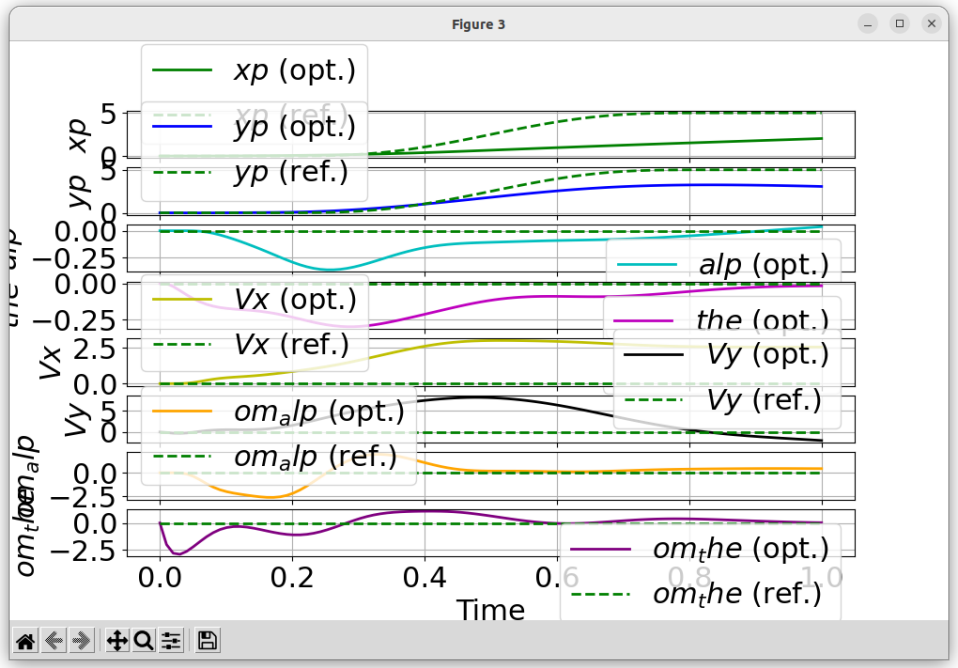

Figure 3.3: Optmal trajectory and desired curve iteration-5

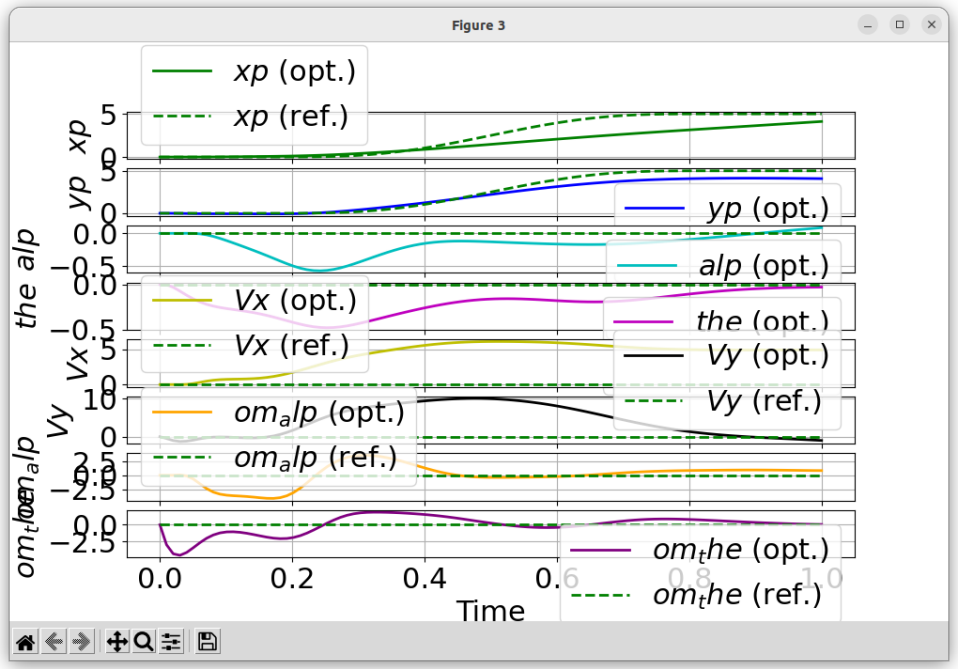

Figure 3.4: Optmal trajectory and desired curve iteration-7

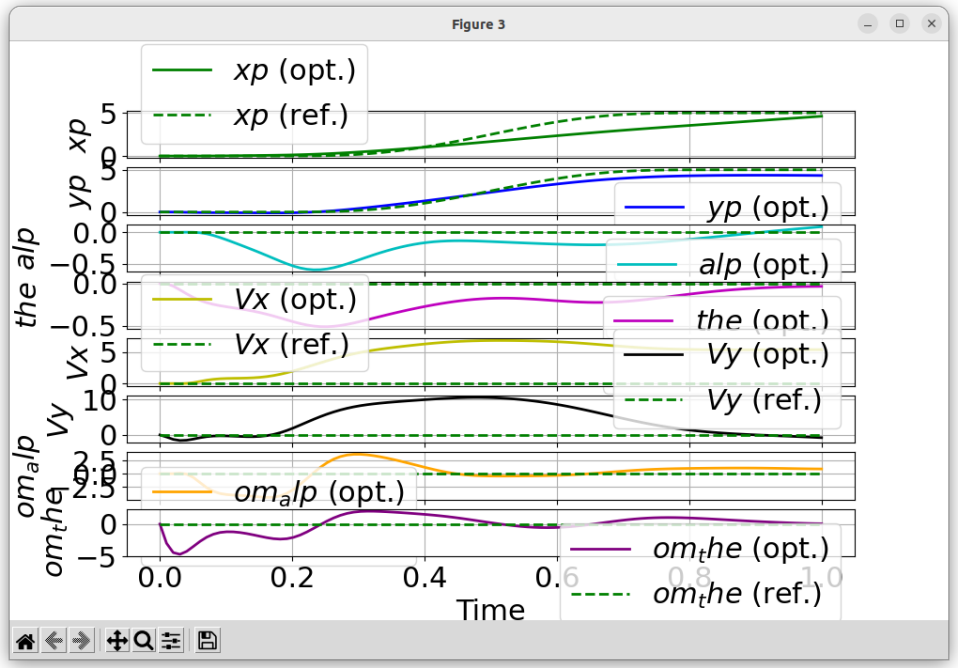

Figure 3.5: Optmal trajectory and desired curve iteration-9

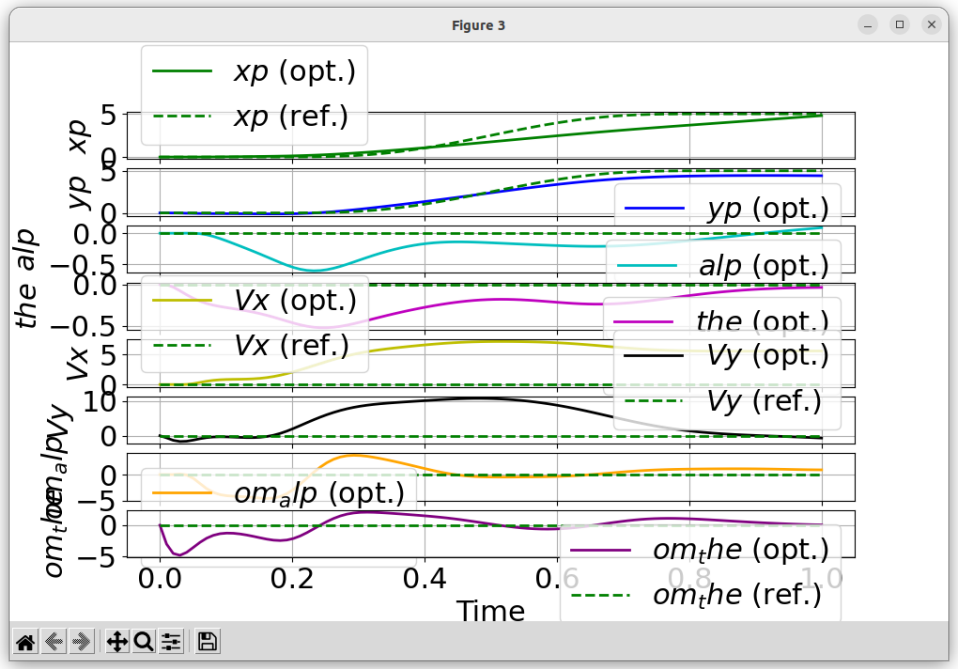

Figure 3.6: Optmal trajectory and desired curve iteration-10

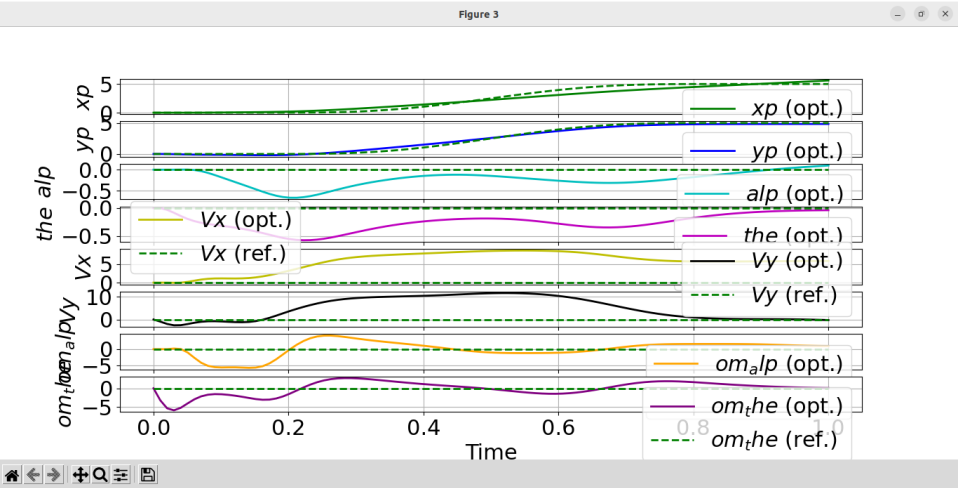

Figure 3.7: Optmal trajectory and desired curve fianl

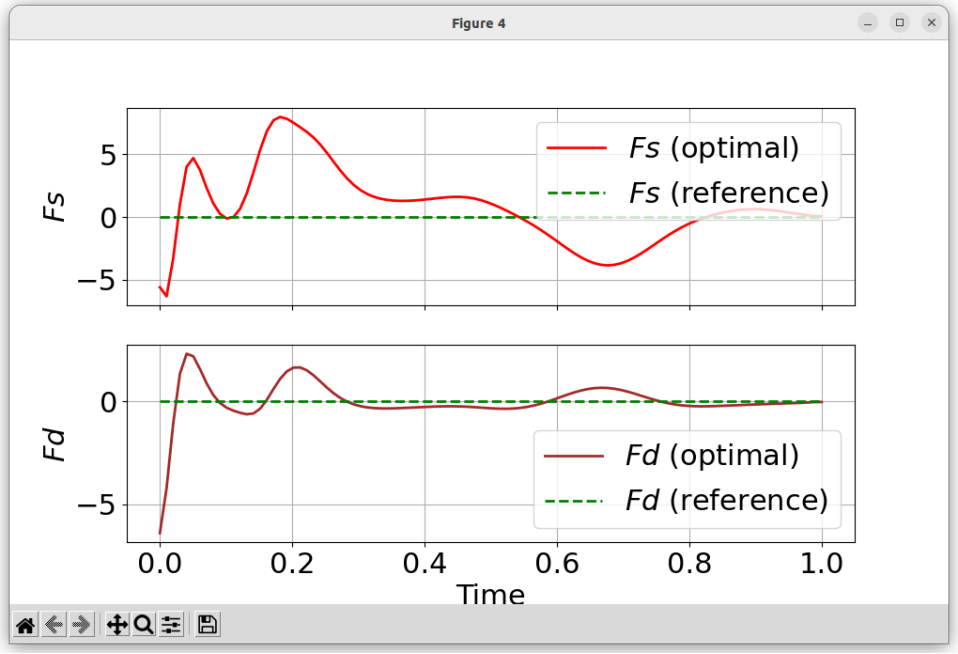

Figure 3.8: Optimal trajectory for inputs and desired input curve

As shown in Figure 3.1-3.8: the dotted lines represent the bell-shaped trajectory that we defined (the reference trajectory) and the solid lines represent the optimal trajectory computed by the Newton's Method.

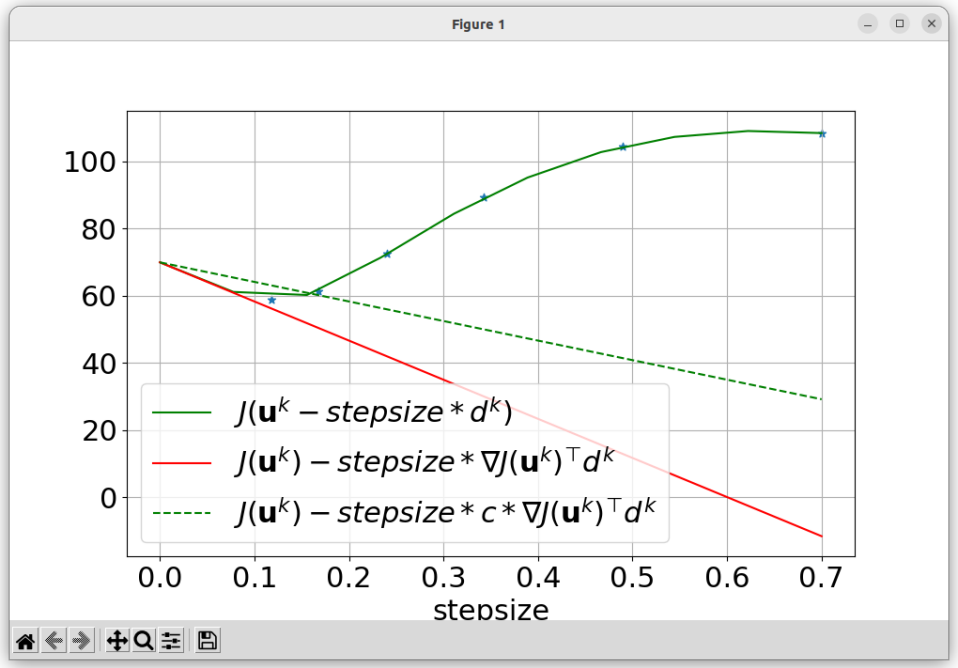

Figure 3.9: Armijo descent direction plot iteration-1

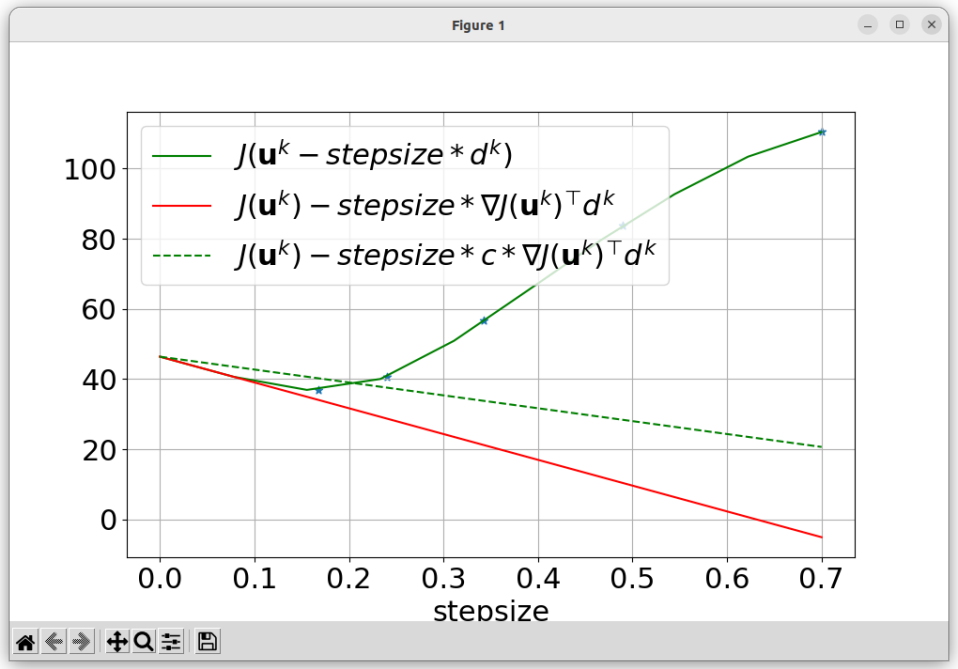

Figure 3.10: Armijo descent direction plot iteration-3

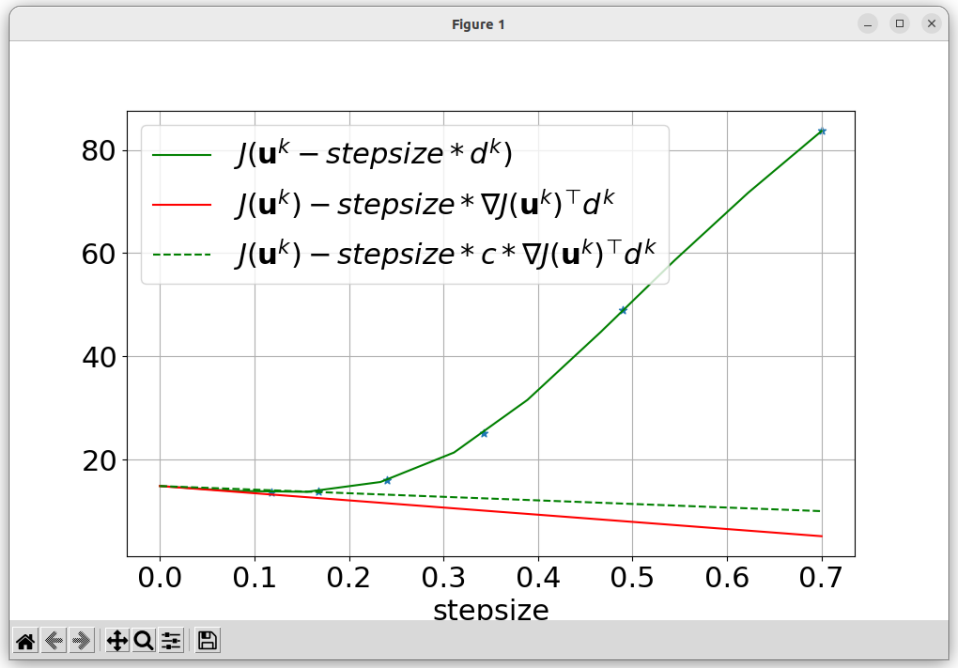

Figure 3.11: Armijo descent direction plot iteration-5

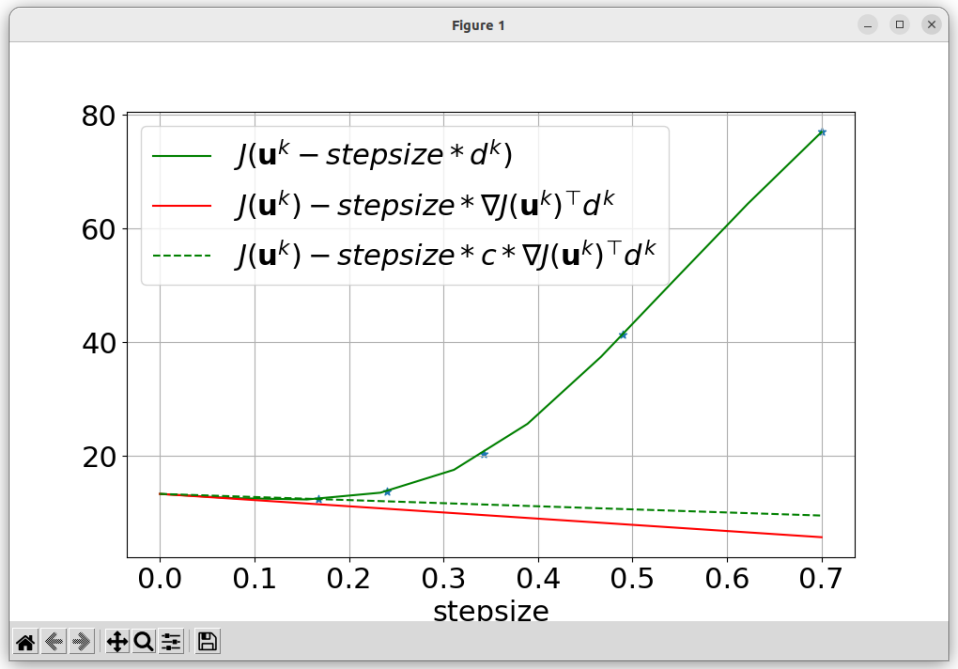

Figure 3.12: Armijo descent direction plot iteration-7

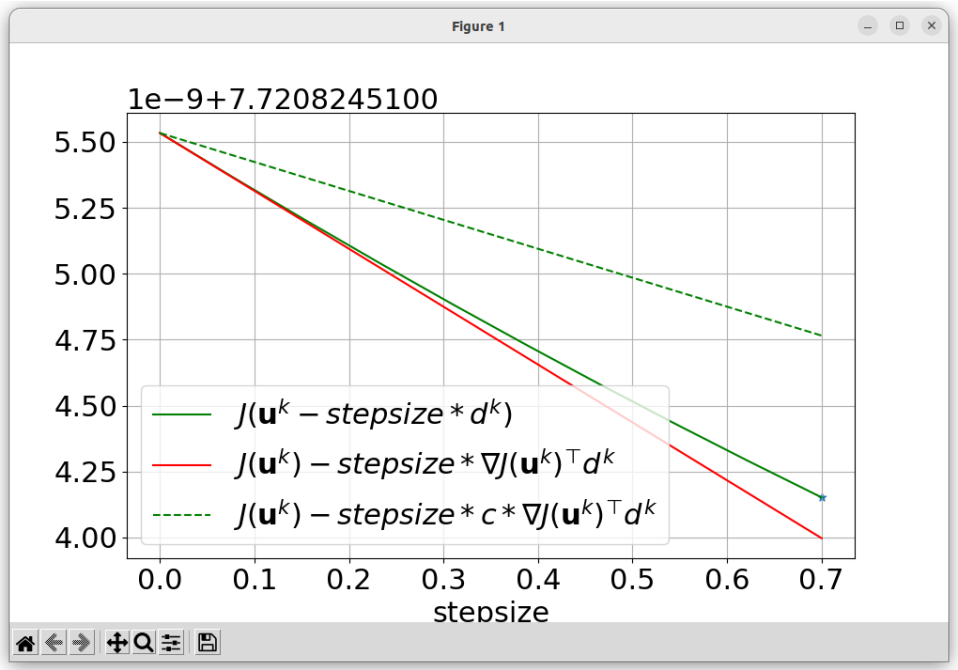

Figure 3.13: Armijo descent direction plot final

The final Armijo's selection rule and its iterations for the our smooth function are shown in Figure 3.9-3.13.

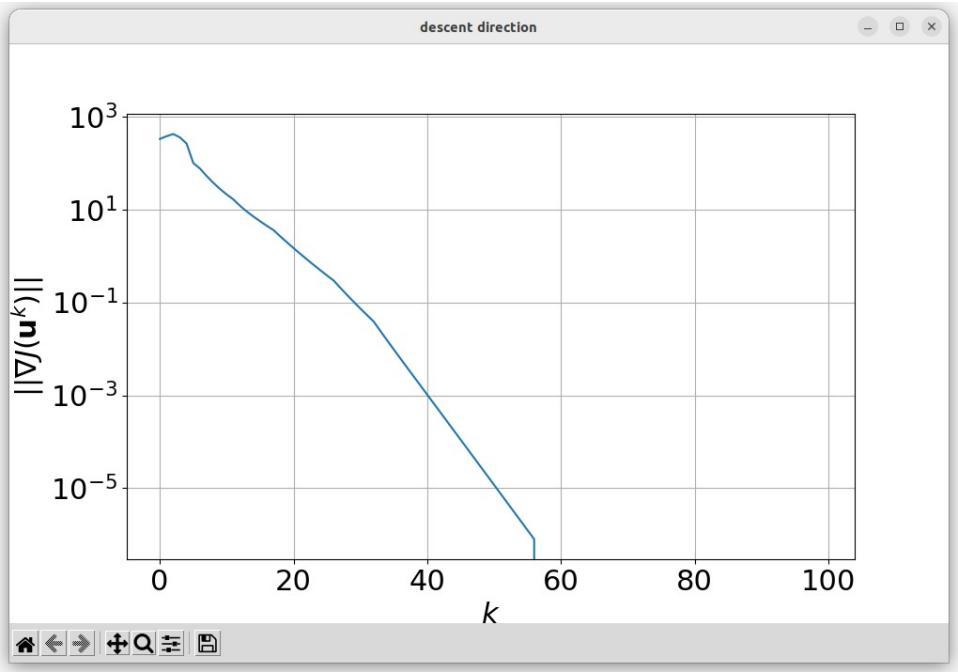

Figure 3.14: Norm of the Descent Direction

Figure 3.14 illustrates the descent direction norm for the Bell-shaped trajectory. In this particular test, we established a maximum iteration limit of 100. As depicted in the plot, the descent direction norm consistently diminishes throughout the iterations. This observation indicates that our optimizer is functioning appropriately, steadily approaching the minimum.

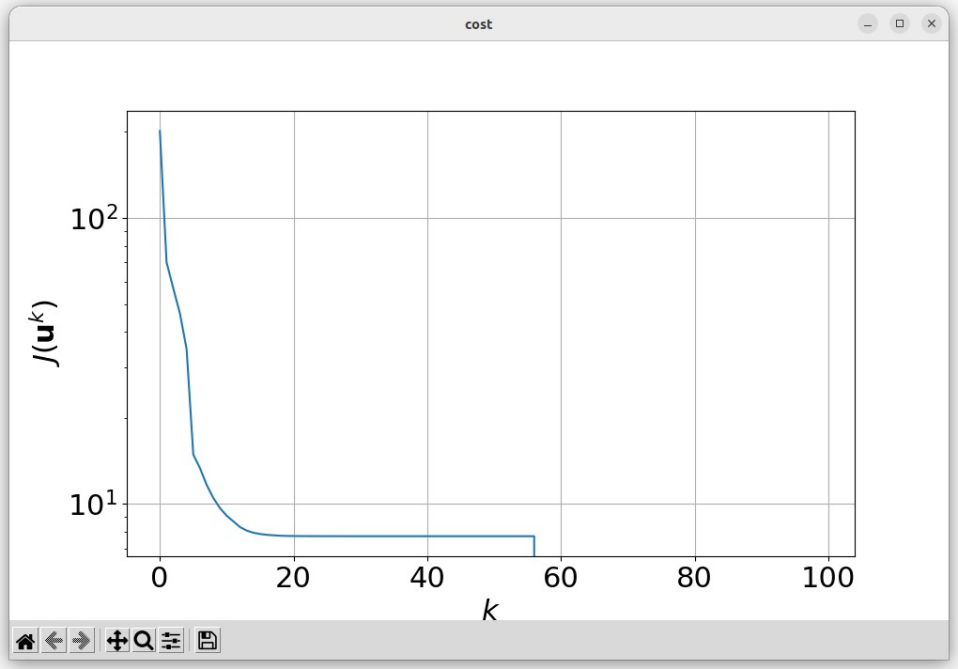

Figure 3.15: Cost of the Bell-shaped Trajectory

The Cost Function continuously decreases over the course of all 56 iterations, which is a positive outcome given that our objective is to minimize it.

Chapter 4

Task 3 - Trajectory tracking via LQR

4.1 Tracking via LQR

Idea: We want to use a (stablizing) feedback Linear Quadratic Regulator (LQR) on the linearization to track the optimal trajectory (x^{opt}, u^{opt}).

Step 1: Linearize the dynamics about the optimal trajectory (x^{opt}, u^{opt}) to get the LTV system

$$\Delta x_{t+1} = A_t^{opt} \Delta x_t + B_t^{opt} \Delta u_t$$

where $A_t^{opt} \in \mathbb{R}^{n \times n}$ and $B_t^{opt} \in \mathbb{R}^{n \times m}$ are defined as:

$$A_t^{opt} = \nabla_1 f(x_t^{opt}, u_t^{opt})^\top$$
$$B_t^{opt} = \nabla_2 f(x_t^{opt}, u_t^{opt})^\top$$

Step 2: Calculate the LQ optimal feedback controller

Solving the optimal control problem

$$\min_{\substack{\Delta x_1, ..., \Delta x_T \\ \Delta u_0, ..., \Delta u_T - 1}} \sum_{t=0}^{T-1} \Delta x_t^\top Q_t^{reg} \Delta x_t + \Delta u_t^\top R_t^{reg} \Delta u_t + \Delta x_T^\top Q_T^{reg} \Delta x_T$$

$$\text{subj. to } \Delta x_{t+1} = A_t^{opt} \Delta x_t + B_t^{opt} \Delta u_t \qquad t = 0, ..., T-1$$
$$\Delta x_0 = 0$$

for some cost matrices $Q_t^{reg} \geq 0 \in \mathbb{R}^{n \times n}$, $^{reg}_{,t} > 0 \in \mathbb{R}^{n \times m}$ and $Q_T^{reg} \geq 0 \in \mathbb{R}^{n \times n} (degree\, of\, freedom)$.

Set $P_T = Q_T^{reg}$ and backward iterate t = {T-1, ..., 0}:

$$P_t = Q_t^{reg} + {A_t^{opt}}^\top P_{t+1} A_t^{opt} - ({A_t^{opt}}^\top P_{t+1} B_t^{opt})(R_t^{reg} + {B_t^{opt}}^\top P_{t+1} B_t^{opt})^{-1}({B_t^{opt}}^\top P_{t+1} A_t^{opt})$$

and define for all t = {0, ..., T-1}, compute the feedback gain $K_t^{reg} \in \mathbb{R}^{m \times n}$

$$K_t^{reg} := -(R_t^{reg} + {B_t^{opt}}^\top P_{t+1} B_t^{opt})^{-1}({B_t^{opt}}^\top P_{t+1} A_t^{opt})$$

Step 3: Track the generated (optimal) trajectory

We apply the feedback controller designed on the linearization to the nonlinear system to track (x^{opt}, u^{opt}). Namely for all t = {0, ..., T-1}, we apply

$$u_t = u_t^{opt} + K_t^{reg}(x_t - x_t^{opt})$$

$$x_{t+1} = f(x_t, u_t)$$

$$\text{with } x_0 = 0$$

Finally we get the tracked trajectory (x^{lqr}, u^{opt}).

4.2 Tracked Trajectories

The controller monitors the trajectories indicated by the red dotted lines, while the blue lines represent the reference (optimal) trajectories.

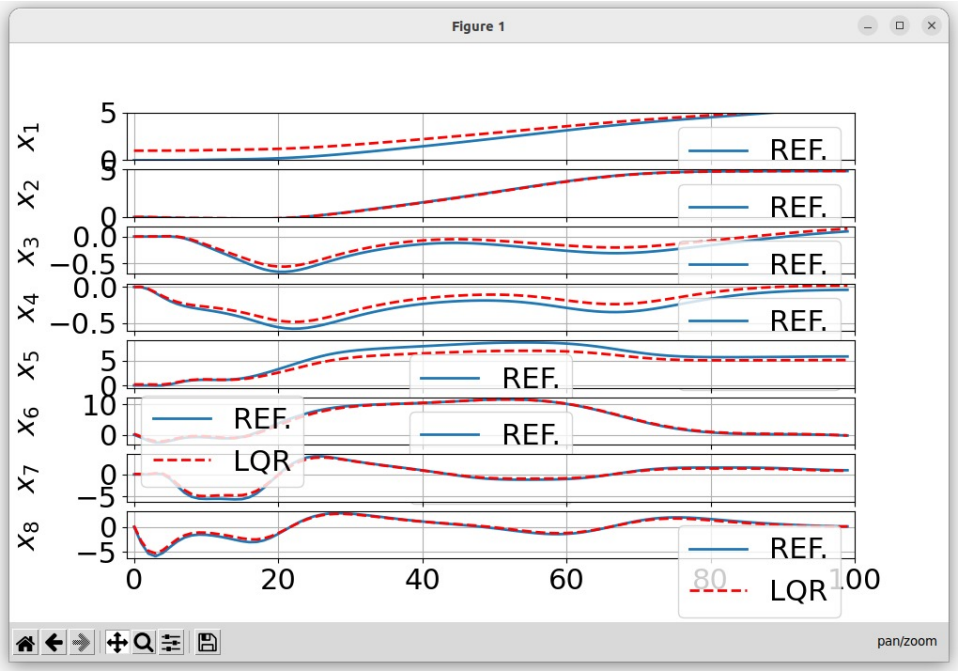

Figure 4.1: State Trajectory Tracking via LQR

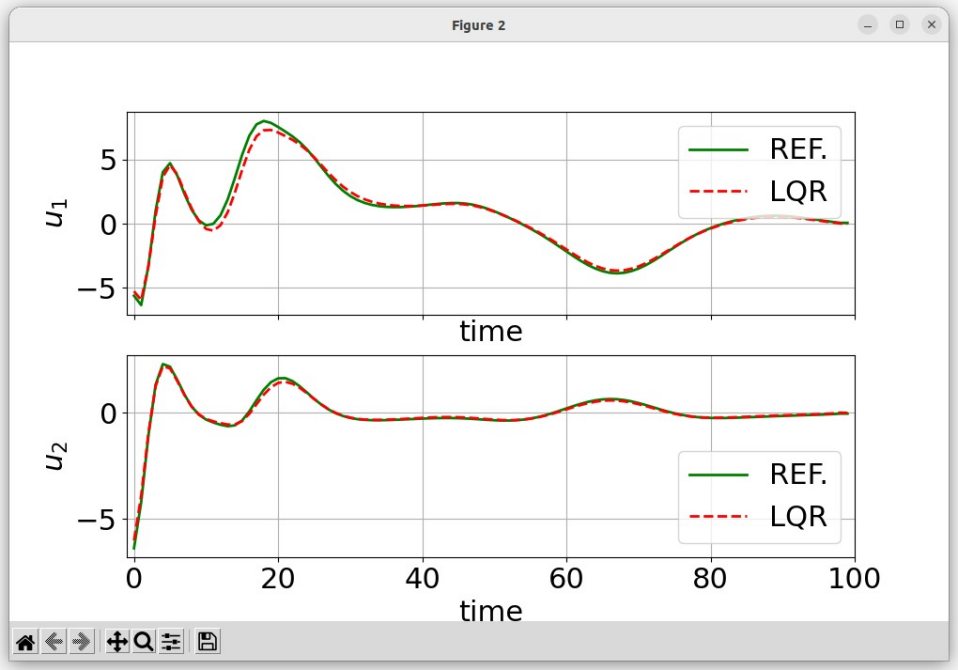

Figure 4.2: Input Trajectory Tracking via LQR

- From these plots, it is evident that the reference trajectories x_1 and

x_2 are accurately adhered to. This outcome is positive, considering that these trajectories correspond to the coordinates of the quadrotor's center of mass, reflecting the effective motion of the quadrotor as depicted in the **animation**.

- The remaining variables assume their optimal values based on the provided reference and the dynamics involved (trade-off).

Chapter 5

Task 4 - Trajectory tracking via MPC

5.1 Tracking via MPC

Idea: For each time instant t, we measure the current state x_t, and then compute the optimal trajectory $x^*_{t|t}, ..., x^*_{t+T|t}, u^*_{t|t}, ..., u^*_{t+T-1|t}$ with initial condition x_t. After that, we need to apply the first control input $u^*_{t|t}$. Finally we measure x_{t+1} and repeat the previous steps to achieve trajectory tracking.

Solve the optimal control problem at each t

At each time instant t, solve the problem:

$$\min_{\substack{x_t,...,x_t+T \\ u_t,...,u_t+T-1}} \sum_{\tau=t}^{t+T-1} l_\tau(x_\tau, u_\tau) + l_{t+T}(x_{t+T})$$

$$\text{subj. to } x_{\tau+1} = f(x_\tau, u_\tau) \quad \forall \tau = t, ..., t+T-1$$

$$\mathrm{x}_\tau \in \chi, u_\tau \in \mu \quad \forall \tau = t, ..., t+T$$

$$x_t = x_t^{meas}$$

where

$$x_{\tau+1} = f(x_\tau, u_\tau)\, is\, the\, so-called\, prediction\, model$$

$$x_t^{meas}\, state\, (of\, the\, real\, system)\, measured\, at\, t$$

MPC: Linear Quadratic Case

Now for our project we need to consider the case where dynamics is linear and the cost quadratic. At each time t, with initial measured state x_t^{meas}, we solve the LQ problem:

$$\min_{\substack{x_t,\dots,x_t+T \\ u_t,\dots,u_t+T-1}} \sum_{t=0}^{t+T-1} x_\tau^\top Q_\tau x_\tau + u_\tau^\top R_\tau u_\tau + x_{t+T}^\top Q_T x_{t+T}$$

$$\text{subj. to } x_{\tau+1} = A_\tau x_\tau + B_\tau u_\tau \quad \forall \tau = t, ..., t+T-1$$

$$\mathrm{x}_\tau \in \chi, u_\tau \in \mu \quad \forall \tau = t, ..., t+T$$

$$x_t = x_t^{meas}$$

where T is the prediction horizon

$$x_\tau \in \mathbb{R}^n \, and \, u_\tau \in \mathbb{R}^m$$

$$A_\tau \in \mathbb{R}^{n\times n}, B_\tau \in \mathbb{R}^{n\times m} \, prediction \, model$$

$$\chi \, connected \, and \, closed, \mu \, compact$$

$$Q_\tau \in \mathbb{R}^{n\times n} \, and \, Q_\tau = Q_\tau^\top \geq 0 \, for \, all \, \tau$$

$$R_\tau \in \mathbb{R}^{m\times m} \, and \, R_\tau = R_\tau^\top > 0 \, for \, all \, \tau$$

$$Q_T \in \mathbb{R}^{n\times n} \, and \, Q_T = Q_T^\top \geq 0$$

5.2 Tracked Trajectories

The controller monitors the trajectories indicated by the red dotted lines, while the blue lines represent the reference (optimal) trajectories.

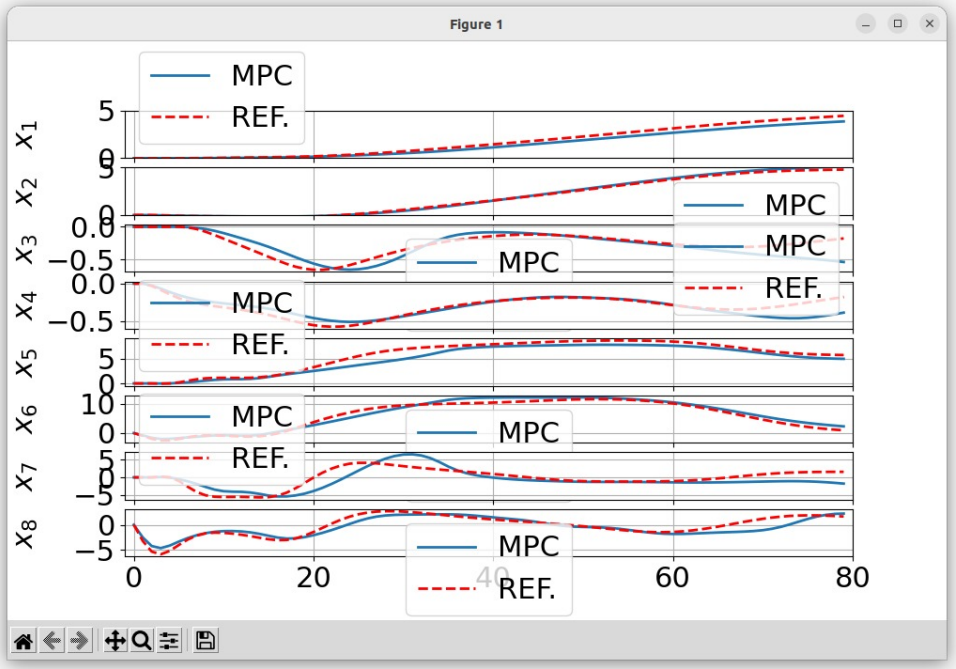

Figure 5.1: State Trajectory Tracking via MPC

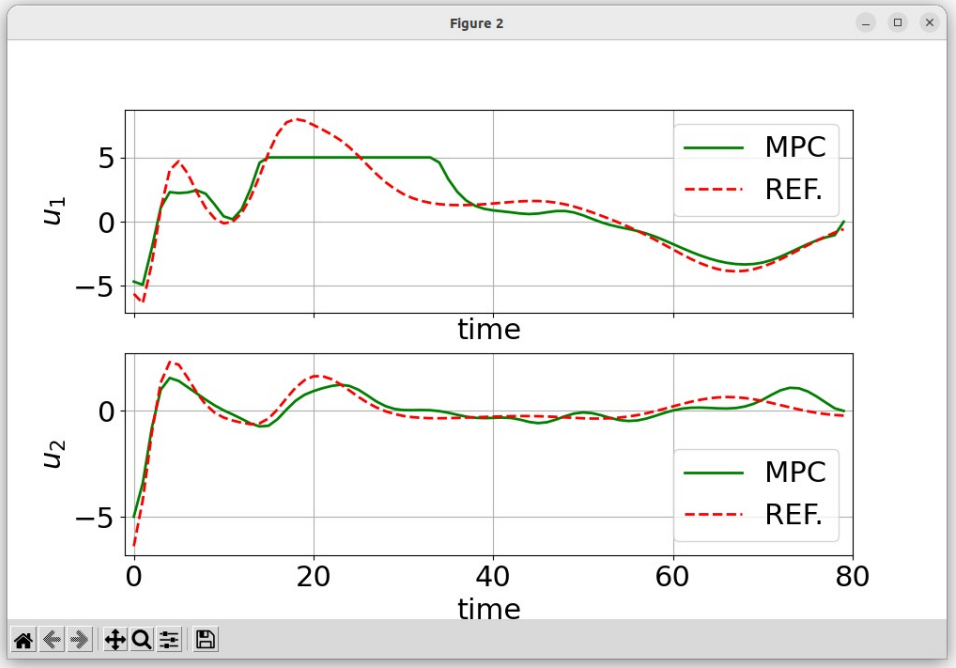

Figure 5.2: Input Trajectory Tracking via MPC

- The remaining variables assume their optimal values based on the

provided reference and the dynamics involved (trade-off).

Conclusions

This project involves employing Newton's Method to generate the optimal trajectory necessary for transitioning from one equilibrium state to another within a discrete non-linear model. The implementation is carried out using Python and comprises three main steps: first, discretizing the dynamic equations; second, establishing a reference trajectory between two flying altitudes; and third, applying Newton's Method to determine the optimal trajectory for the quadrotor to follow. Subsequently, the system is linearized, and feedback controllers as well as MPC algorithms are developed to facilitate tracking of the resulting optimal trajectory.